

Amostragem e Quantização

Processamento de Imagens

Prof. Diógenes Furlan

Sumário

- Textos no GLUT
- Menus no GLUT
- Algoritmos de Quantização
- Algoritmos de Amostragem

TEXTO NO GLUT

glutBitmapCharacter

- Uma das fontes suportadas pela GLUT é a *bitmap*, onde cada caractere corresponde a um *bitmap* que é gerado com a função *glBitmap*.
 - A função *glutBitmapCharacter* exibe um caractere deste tipo usando OpenGL.
- Os parâmetros de entrada desta função são: **(void *font, int character)**.
- As fontes disponíveis são:
 - GLUT_BITMAP_8_BY_13
 - GLUT_BITMAP_9_BY_15
 - GLUT_BITMAP_TIMES_ROMAN_10
 - GLUT_BITMAP_TIMES_ROMAN_24
 - GLUT_BITMAP_HELVETICA_10
 - GLUT_BITMAP_HELVETICA_12
 - GLUT_BITMAP_HELVETICA_18

Display

```
void DesenhaTexto(char *buf)
{
    glPushMatrix();
    glRasterPos2f(x,y); //posição do texto
    glColor3f( glWHITEf ); // cor do texto

    while(*buf)
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_18, *buf++);
    glPopMatrix();
}
```

MENU

Menu com o GLUT

- ***glutMotionFunc***: Função *callback* quando *mouse* é movido com um dos seus botões pressionados.
- ***glutPassiveMotionFunc***: Função *callback* quando *mouse* é movido enquanto nenhum de seus botões está pressionado.
- ***glutCreateMenu***: Cria um novo menu *pop-up* e estabelece a função *callback*
- ***glutAddMenuEntry***: Adiciona uma entrada no final do menu corrente.
- ***glutAddSubMenu***: Adiciona um submenu no final do menu corrente.
- ***glutAttachMenu***: Função que "relaciona" um botão do *mouse* com o identificador do menu corrente.
 - GLUT_LEFT_BUTTON
 - GLUT_MIDDLE_BUTTON ou
 - GLUT_RIGHT_BUTTON

Gerencia Mouse

```
// Função callback chamada para gerenciar eventos do mouse
```

```
void GerenciaMouse(int button, int state, int x, int y)
{
    if (button == GLUT_RIGHT_BUTTON)
        if (state == GLUT_DOWN)
            CriaMenu();
    glutPostRedisplay();
}
```

```
// main
```

```
glutMouseFunc(GerenciaMouse);
```

Cria Menu

```
// Criacao do Menu
```

```
void CriaMenu() {  
    int menu,submenu1,submenu2;  
  
    submenu1 = glutCreateMenu(MenuCor);  
    glutAddMenuEntry("Vermelho",0);  
    glutAddMenuEntry("Verde",1);  
    glutAddMenuEntry("Azul",2);  
  
    submenu2 = glutCreateMenu(MenuPrimitiva);  
    glutAddMenuEntry("Quadrado",0);  
    glutAddMenuEntry("Triângulo",1);  
    glutAddMenuEntry("Losango",2);  
  
    menu = glutCreateMenu(MenuPrincipal);  
    glutAddSubMenu("Cor",submenu1);  
    glutAddSubMenu("Primitivas",submenu2);  
    glutAttachMenu(GLUT_RIGHT_BUTTON);  
}
```

Menu Principal

```
// Gerenciamento do menu principal
```

```
void MenuPrincipal(int op)  
{  
}
```

Menu Primitiva

// Gerenciamento do menu com as opções das primitivas

```
void MenuPrimitiva(int op)
{
    switch(op) {
        case 0: primitiva = QUADRADO; break;
        case 1: primitiva = TRIANGULO; break;
        case 2: primitiva = LOSANGO; break;
    }
    glutPostRedisplay();
}
```

Menu Cor

// Gerenciamento do menu com as opções das cores

```
void MenuCor(int op)
{
    switch(op) {
        case 0:
            r = 1.0f; g = 0.0f; b = 0.0f;
            break;
        case 1:
            r = 0.0f; g = 1.0f; b = 0.0f;
            break;
        case 2:
            r = 0.0f; g = 0.0f; b = 1.0f;
            break;
    }
    glutPostRedisplay();
}
```

QUANTIZAÇÃO

Copia Imagem

```
void CopyImage()
{
    int x,y;
    unsigned char r,g,b;

    for(x=0; x<Image->getSizeX(); x++) {
        for(y=0; y<Image->getSizeY(); y++)
        {
            Image->ReadPixel(x, y, r,g,b);
            NovaImagem->DrawPixel(x, y, r,g,b);
        }
    }
}
```

Tons de Cinza

```
void ConvertToGrayScale()
{
    int x,y;
    unsigned char r,g,b;

    for(x=0; x<Image->getSizeX(); x++) {
        for(y=0; y<Image->getSizeY(); y++)
        {
            Image->ReadPixel(x, y, r,g,b);
            i = (0.30*r + 0.59*g + 0.11*b);
            NovaImagem->DrawPixel(x, y, i,i,i);
        }
    }
}
```

Preto e Branco

```
void ConvertBlackAndWhite()  
{
```

```
    int x,y;  
    unsigned char r,g,b;
```

Quanto maior o LIMIAR, mais preto tem.

```
    for(x=0; x<Image->getSizeX(); x++) {  
        for(y=0; y<Image->getSizeY(); y++)  
        {
```

```
            Image->ReadPixel(x, y, r,g,b);
```

```
            i = (0.299*r + 0.587*g + 0.114*b);
```

```
            if (i < LIMIAR)
```

```
                NovaImagem->DrawPixel(x, y, 0,0,0);
```

```
            else
```

```
                NovaImagem->DrawPixel(x, y, 255,255,255);
```

```
        }
```

```
    }
```

```
}
```

Canal Verde

```
void GreenChannel()
{
    int x,y;
    unsigned char r,g,b;

    for(x=0; x<Image->getSizeX(); x++) {
        for(y=0; y<Image->getSizeY(); y++)
        {
            Image->ReadPixel(x, y, r,g,b);
            NovaImagem->DrawPixel(x, y, 0,g,0);
        }
    }
}
```

AMOSTRAGEM

Zoom out (Afastar a imagem)

```
void ZoomOut(int fator)
{
    int x,y;
    unsigned char r,g,b;

    for(x=0; x<Image->getSizeX(); x+=fator) {
        for(y=0; y<Image->getSizeY(); y+=fator)
        {
            Image->ReadPixel(x, y, r,g,b);
            NovaImagem->DrawPixel(x/fator, y/fator, r,g,b);
        }
    }
}
```

Thumbnail

```
void Thumbnail(int sizeX, sizeY)
{
    int x,y;
    unsigned char r,g,b;
    int fatorX = Image->getSizeX() / sizeX;
    int fatorY = Image->getSizeY() / sizeY;

    for(x=0; x<sizeX; x++) {
        for(y=0; y<sizeY; y++)
        {
            Image->ReadPixel(x*fatorX, y*fatorY, r,g,b);
            NovaImagem->DrawPixel(x, y, r,g,b);
        }
    }
}
```

EXERCÍCIOS

Exercícios

1. Fazer um algoritmo para reduzir quantização de 256 para 16 tons de cinza.
2. Faça um algoritmo para gerar uma imagem apenas com o canal vermelho do RGB.
3. É possível, com o aprendizado até o momento, fazer um algoritmo para dizer se a imagem é binária, monocromática ou colorida?